Viva answers for questions

Lp 2 1-5.zip
Zip Archive

analyze the zip and give oral related qustions

Here are oral/viva-style questions based on the contents of your ZIP file (Assignments 1–5: HTML, CSS, JS, PHP, MySQL).

✅ Oral/Viva Questions Based on Each Assignment

📂 Assignment 1 — Basic HTML & CSS Website

Files: index.html, about.html, events.html, contact.html, style.css

Oral Questions

1. What is the purpose of the `<head>` and `<body>` sections in HTML?
2. How do you link a CSS file to an HTML document?
3. Explain the difference between block-level and inline elements.
4. What are semantic HTML tags? Can you name a few?
5. How does the CSS box model work?
6. What is the difference between `id` and `class` selectors in CSS?
7. How do you make a navigation bar in HTML/CSS?
8. What are HTML attributes? Give examples.
9. Explain how you would make a responsive webpage.
10. What is the use of `<meta>` tags?

📂 Assignment 2 — HTML & CSS Layout

Files: index.html, style2.css

Oral Questions

1. What is Flexbox? When would you use it?
2. Explain the difference between margin and padding.
3. How do CSS position values (static, relative, absolute, fixed) work?

4. What is the purpose of `z-index`?
5. How do you center a div horizontally and vertically?
6. What is the difference between internal, external, and inline CSS?
7. How do media queries work in CSS?
8. What is the difference between `display: block`, `inline`, and `inline-block`?
9. How does CSS specificity work?
10. What are pseudo-classes? Give examples.

📂 Assignment 3 — HTML Form + JavaScript Validation

Files: register.html, script3.js, style3.css

Oral Questions

1. What is the purpose of client-side validation?
2. Difference between `==` and `===` in JavaScript.
3. How do you access HTML elements in JavaScript?
4. What is `addEventListener` used for?
5. Explain the use of `preventDefault()` in forms.
6. What is DOM manipulation? Examples?
7. How do you validate email or phone numbers?
8. What happens if JavaScript validation is disabled by the user?
9. What are JavaScript data types?
10. Explain scope in JavaScript (var, let, const).

📂 Assignment 4 — JavaScript Events

Files: events.html, script4.js, style4.css

Oral Questions

1. What are JavaScript events? Give examples.
2. Difference between inline event handlers and external JS event listeners.
3. What does event bubbling mean?
4. What is event delegation?
5. How does `onclick` differ from `addEventListener`?
6. What is the event object and what does it contain?
7. What is DOMContentLoaded?

8. Explain callbacks with an example.
9. How do timers (`setTimeout`, `setInterval`) work in JavaScript?
10. What are arrow functions and how do they differ from regular functions?

📂 Assignment 5 — HTML Form + PHP + MySQL

Files: register.html, save.php, mysql table.txt, style5.css

Oral Questions

1. What is the purpose of PHP in web development?
2. Explain how a form sends data to a PHP script.
3. What is the difference between `GET` and `POST`?
4. How do you connect PHP to a MySQL database?
5. What are SQL injection attacks and how do you prevent them?
6. Explain how `$_POST` and `$_GET` work.
7. What is server-side validation?
8. What is a primary key in MySQL?
9. How do you insert form data into a database using PHP?
10. What are prepared statements and why are they important?

🎯 If you want, I can also generate:

✔️ Viva answers
✔️ A full mock interview
✔️ Multiple-choice or coding questions
✔️ Corrections or improvements to the code in the ZIP

Would you like that?

give the viva ans for each questions

Below are **clear, viva-style answers** for **ALL the questions** from Assignments 1–5.
(Short, simple, to-the-point answers — perfect for oral exams.)

✅ Assignment 1 – Viva Answers (Basic HTML & CSS)

1. Purpose of `<head>` and `<body>`?

- `<head>` contains metadata, title, links to CSS/JS, not visible to users.
- `<body>` contains all visible content like text, images, forms, etc.

2. How to link CSS in HTML?

```html
<link rel="stylesheet" href="style.css">
```

3. Block vs Inline elements?

- **Block**: takes full width, starts new line (e.g., `<div>`, `<p>`).
- **Inline**: takes only needed width (e.g., `<span>`, `<a>`).

4. Semantic HTML? Examples?

Tags that describe meaning: `<header>`, `<footer>`, `<article>`, `<section>`.

5. CSS Box Model?

Every element consists of: **content → padding → border → margin**.

6. Difference between id and class?

- **id**: unique, used once.
- **class**: reusable on multiple elements.

7. How to create a navigation bar?

Using `<nav>` + `<ul>` + CSS Flexbox or inline-block styling.

8. What are HTML attributes?

Extra information for elements (e.g., `href`, `src`, `alt`, `id`, `class`).

9. How to make a responsive page?

Using **media queries**, **flexbox**, **grid**, and **viewport meta tag**.

10. Purpose of `<meta>` tags?

Provides metadata like charset, viewport, SEO keywords, description.

✅ Assignment 2 – Viva Answers (CSS Layout)

1. What is Flexbox?

A one-dimensional layout system used to align items easily.

2. Margin vs Padding?

- **Margin**: space *outside* an element.
- **Padding**: space *inside* around the content.

3. CSS position types?

- **static** – default
- **relative** – moves relative to normal position
- **absolute** – positioned relative to the nearest positioned parent
- **fixed** – stays fixed on screen
- **sticky** – switches between relative and fixed

4. Purpose of `z-index`?

Controls stacking order of overlapping elements.

5. How to center a div?

```css
margin: auto;
```

OR using Flexbox:

```css
display: flex;
justify-content: center;
align-items: center;
```

6. Types of CSS?

- Inline
- Internal
- External (most recommended)

7. What are media queries?

CSS rules that apply only under certain screen sizes.

8. Block vs Inline vs Inline-Block?

- Block: full width
- Inline: no width/height control
- Inline-block: inline but can set width/height

9. CSS Specificity?

Priority rules:
Inline > ID > Class > Tag.

10. What are pseudo-classes?

Special states, e.g., `:hover`, `:focus`, `:active`.

✅ Assignment 3 – Viva Answers (JavaScript + Form Validation)

1. Purpose of client-side validation?

To catch errors early and reduce server load.

2. `==` vs `===`?

- `==` checks value only.
- `===` checks value + type.

3. How to access HTML elements?

Using methods like:
`getElementById()`, `querySelector()`.

4. Purpose of `addEventListener`?

To attach multiple events in a clean way.

5. What is `preventDefault()`?

Stops default form submission.

6. What is DOM manipulation?

Changing HTML dynamically using JS (e.g., updating text, styles).

7. How to validate email/phone?

Using regex or built-in HTML attributes.

8. What if JS validation is disabled?

Server-side validation must handle it for security.

9. JavaScript data types?

String, Number, Boolean, Object, Undefined, Null, Symbol, BigInt.

10. Scope (`var`, `let`, `const`)?

- `var`: function-scoped
- `let`: block-scoped
- `const`: block-scoped, cannot reassign

✅ Assignment 4 – Viva Answers (JavaScript Events)

1. What are JS events?

Actions like click, keypress, mouseover that trigger code.

2. Inline vs external event handling?

- Inline uses HTML attributes.
- External uses JS and is cleaner, reusable.

3. Event bubbling?

Event travels from inner → outer elements.

4. Event delegation?

Handling events on a parent instead of each child.

5. `onclick` vs `addEventListener`?

- `onclick` → only one handler
- `addEventListener` → multiple handlers allowed

6. What is the event object?

Contains info like target element, coordinates, keys pressed.

7. What is DOMContentLoaded?

Fires when the HTML is fully loaded (before images).

8. What is a callback?

A function passed as an argument to another function.

9. How do timers work?

- `setTimeout()` runs once after delay.
- `setInterval()` repeats continuously.

10. Arrow functions vs regular functions?

Shorter syntax and do not have their own `this`.

✅ Assignment 5 – Viva Answers (PHP + MySQL)

1. Purpose of PHP?

Server-side scripting to process data and interact with databases.

2. How does a form send data to PHP?

Using `action` URL and method (`POST` / `GET`).

3. GET vs POST?

- GET: visible in URL, less secure
- POST: hidden, more secure, larger data

4. How to connect PHP to MySQL?

Using `mysqli_connect()` or PDO.

5. What is SQL Injection?

A hacking method where SQL code is injected into queries.
Prevention: prepared statements.

6. What are `$_POST` and `$_GET`?

Superglobal arrays to receive form data.

7. What is server-side validation?

Validation performed on the server to ensure security.

8. What is a primary key?

A unique identifier for each row.

9. How to insert form data in MySQL?

Using `INSERT INTO` SQL query in PHP.

10. What are prepared statements?

Queries that separate code from data to prevent SQL injection.

🎯 Want me to generate a PDF of all questions + answers for your viva?

Or do you want **additional expected viva questions**?

explain in detailed answer

Great — I'll expand each viva answer into a clear, detailed explanation you can use in an oral exam. I'll keep each answer focused, give examples where useful, and include short code snippets for clarity. I'll go assignment by assignment.

✅ Assignment 1 — Detailed Answers (HTML & CSS)

1. What is the purpose of the `<head>` and `<body>` sections in HTML?

`<head>` holds metadata and resources needed by the browser or search engines but not directly rendered as page content. Typical contents: `<title>` (page title shown in the browser tab), `<meta>` tags (charset, viewport, description), links to CSS (`<link>`), and scripts that should load early.
`<body>` contains the content visible to users — headings, paragraphs, images, forms, navigation, etc. The browser turns the body's elements into the visual page.
Separating them improves performance and content semantics.

2. How do you link a CSS file to an HTML document?

Use a `<link>` inside `<head>`:

```html
<link rel="stylesheet" href="style.css">
```

`rel="stylesheet"` tells the browser the resource is CSS; `href` is the file path. You can also use `<style>` for embedded CSS or the `style` attribute for inline styles, but external CSS is best for maintainability and caching.

3. Explain the difference between block-level and inline elements.

Block-level elements (e.g., `<div>`, `<p>`, `<h1>`) start on a new line, expand to the container's full width by default, and can hold other block or inline elements. **Inline** elements (e.g., `<span>`, `<a>`, `<strong>`) flow inside a line of text, take only the space they need, and cannot contain block elements. Understanding the difference is essential for layout and spacing.

4. What are semantic HTML tags? Can you name a few?

Semantic tags convey meaning about the content to browsers and assistive technologies (screen readers) and improve SEO. Examples: `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`. Using semantics clarifies document structure and makes maintenance easier.

5. How does the CSS box model work?

Every element is a rectangular box composed of (from inside out): **content → padding → border → margin**. Content holds text/images; padding adds inner spacing; border surrounds padding; margin separates the element from others. Box-sizing controls whether width/height includes padding and border (`box-sizing: border-box` is common, it makes sizing more intuitive).

6. What is the difference between `id` and `class` selectors in CSS?

An `id` is meant to be unique on a page (one element) and selected with `#idname`. A `class` can be reused on many elements and selected with `.classname`. Specificity-wise, `id` selectors are stronger than class selectors. Use `id` for single unique elements, `class` for reusable styling.

7. How do you make a navigation bar in HTML/CSS?

Common structure:

```html
<nav>
 <ul class="nav">
  <li><a href="/">Home</a></li>
```

```
    <li><a href="/about">About</a></li>
  </ul>
</nav>
```

Then use CSS with flexbox or grid to lay items horizontally, add spacing, hover styles, and responsive behavior (collapse to a hamburger menu on small screens). Semantic `<nav>` helps accessibility.

8. What are HTML attributes? Give examples.

Attributes provide additional information or behavior to elements. Examples: `href` on `<a>`, `src` and `alt` on `<img>`, `id`, `class`, `type` on `<input>`, `action` and `method` on `<form>`. Attributes are key-value pairs inside the tag and can affect rendering, behavior, or accessibility.

9. Explain how you would make a responsive webpage.

Key techniques: the viewport meta tag (`<meta name="viewport" content="width=device-width, initial-scale=1">`), flexible layouts with percentages/flexbox/grid, responsive images (`max-width:100%; height:auto`), and CSS media queries to change styles at breakpoints. Progressive enhancement and testing across devices are important. Aim to adapt layout and typography rather than simply shrinking content.

10. What is the use of `<meta>` tags?

`<meta>` tags supply metadata: charset (`<meta charset="utf-8">`), viewport for responsiveness, description for SEO, author, and robots directives. They don't render content but inform browsers, search engines, and social media scrapers how to treat the page.

✅ Assignment 2 — Detailed Answers (CSS Layout)

1. What is Flexbox? When would you use it?

Flexbox is a one-dimensional layout model (row or column) designed to distribute space and align items within a container. Use it for navigation bars, card rows, aligning elements vertically/horizontally, and flexible, responsive components. Key properties: `display: flex`, `justify-content`, `align-items`, `flex-grow`, `flex-shrink`, and `flex-basis`.

2. Explain the difference between margin and padding.

Padding is inner space between content and border; it enlarges the element's clickable/visible area. **Margin** is outer space that separates an element from siblings. Padding affects background and border; margin does not. For example, increasing padding makes the element bigger; increasing margin moves it away from others.

3. How do CSS position values (static, relative, absolute, fixed, sticky) work?

- **static**: default, normal flow.
- **relative**: offset relative to normal position without affecting flow of other elements.
- **absolute**: positioned relative to nearest ancestor that's positioned (non-static); removed from normal flow.
- **fixed**: positioned relative to viewport (stays on screen during scroll).
- **sticky**: behaves like `relative` until crossing a scroll threshold, then behaves like `fixed`.
  These let you control layering and placement, but absolute/fixed remove elements from document flow, which affects layout.

4. What is the purpose of `z-index`?

`z-index` sets stacking order for positioned elements (elements with position other than static). Higher `z-index` appears above lower values. It only works when elements are positioned (`relative`, `absolute`, `fixed`, `sticky`). Useful for overlays, modals, and dropdowns.

5. How do you center a div horizontally and vertically?

Several ways:

- Flexbox:

```
css

.container { display:flex; justify-content:center; align-items:center; }
```

- For a fixed-size block: `margin: auto` for horizontal centering; for vertical centering use Flexbox or CSS Grid. Older methods include absolute positioning with transforms:

```
css
```

```
.child { position: absolute; left:50%; top:50%; transform: translate(-50%, -50%); }
```

6. What is the difference between internal, external, and inline CSS?

- **External**: separate `.css` file linked via `<link>` — best for reuse and caching.
- **Internal**: `<style>` inside `<head>` — page-specific styles.
- **Inline**: `style="..."` on the element — highest specificity, not reusable, used sparingly. External is preferred for maintainability.

7. How do media queries work in CSS?

Media queries apply styles conditionally based on device characteristics (width, height, resolution). Example:

```
css

@media (max-width: 600px) {
  .nav { display: none; }
}
```

They're fundamental for responsive design, allowing different layouts at breakpoints.

8. What is the difference between `display:block`, `inline`, and `inline-block`?

- `block`: element starts on new line; can set width/height.
- `inline`: flows with text; cannot set width/height.
- `inline-block`: flows inline but allows width/height. `inline-block` is handy for horizontally aligned elements that need sizing.

9. How does CSS specificity work?

Specificity determines which rule applies when multiple rules target the same element. Simplified order: inline styles > IDs > classes/attributes/pseudo-classes > element selectors/pseudo-elements. If specificity ties, the last declared rule wins. Avoid over-reliance on `!important` — instead write clear selectors.

10. What are pseudo-classes? Give examples.

Pseudo-classes define a special state of an element, e.g., `:hover` (mouse over), `:focus` (input focused), `:active` (being clicked), `:nth-child(2)` (position-based), `:visited`. They allow styling based on state or position without additional markup.

✅ Assignment 3 — Detailed Answers (JS + Form Validation)

1. What is the purpose of client-side validation?

Client-side validation provides immediate feedback to users (e.g., required fields, format errors), improving UX and reducing network round-trips. However, it's not secure by itself because users can bypass it; server-side validation is still required as the authoritative gatekeeper.

2. Difference between `==` and `===` in JavaScript.

`==` does type-coercive equality — it converts operands to comparable types before comparing (`'5' == 5` → true). `===` is strict equality — it checks both type and value (`'5' === 5` → false). Use `===` to avoid unexpected coercion bugs.

3. How do you access HTML elements in JavaScript?

Common methods:

- `document.getElementById('id')`
- `document.getElementsByClassName('class')`
- `document.getElementsByTagName('tag')`
- `document.querySelector('.class')` (single)
- `document.querySelectorAll('div.some')` (NodeList)
  `querySelector` / `querySelectorAll` are flexible and CSS-selector-based.

4. What is `addEventListener` used for?

It attaches an event handler to an element without overwriting existing handlers. Example:

```
js

button.addEventListener('click', function (e) { /* do stuff */ });
```

It supports multiple listeners, options like `capture`, and improves separation of concerns (HTML vs. JS).

5. Explain the use of `preventDefault()` in forms.

`event.preventDefault()` stops the browser's default behavior for an event — in forms this prevents submission and page reload so you can validate or submit via AJAX. It's essential when you need to control the flow in JS.

6. What is DOM manipulation? Examples?

DOM manipulation is changing the document structure, content, or style dynamically through JavaScript. Examples: `element.textContent = 'Hello'`, `element.classList.add('active')`, `parent.appendChild(newNode)`, or `element.style.display = 'none'`. Libraries like jQuery used to simplify this; modern vanilla JS covers most cases.

7. How do you validate email or phone numbers?

Common approaches:

- **HTML5 attributes**: `<input type="email">` or `pattern` attribute for regex.
- **Regex**: use a regular expression to validate format. For email, use a reasonable regex (not overly strict — many valid addresses exist). For phone numbers, consider international formats and possibly a library like libphonenumber for robust validation.

8. What happens if JavaScript validation is disabled by the user?

If client-side JS is disabled or bypassed, no client validation runs — the server must still validate all inputs to prevent malformed data and attacks (e.g., SQL injection, XSS). Treat client-side validation as UX improvement only, never as a security measure.

9. What are JavaScript data types?

Primitive types: `String`, `Number`, `Boolean`, `Undefined`, `Null`, `Symbol`, `BigInt`. Non-primitive: `Object` (including Arrays, Functions, Dates). JavaScript is dynamically typed; variables can hold any type and types can change at runtime.

10. Explain scope in JavaScript (`var`, `let`, `const`).

- `var`: function-scoped and hoisted; can lead to unexpected behavior due to hoisting and lack of block scope.
- `let`: block-scoped, not hoisted in the same problematic way, allows reassignment.
- `const`: block-scoped, cannot be reassigned (but objects referenced by const can be mutated). Prefer `let` / `const` in modern code for clearer scoping.

✅ Assignment 4 — Detailed Answers (JS Events)

1. What are JavaScript events? Give examples.

Events are actions or occurrences (user interactions or browser-generated) that code can respond to. Examples: `click`, `submit`, `keydown`, `mousemove`, `load`, `scroll`. Event-driven programming structures UI interactions around these events.

2. Difference between inline event handlers and external JS event listeners.

Inline: `<button onclick="doSomething()">` — mixes behavior with markup and limits to a single handler.
External: `el.addEventListener('click', doSomething)` — keeps JS separate, allows multiple handlers, is more maintainable and testable.

3. What does event bubbling mean?

Bubbling is when an event starts at the deepest target element and propagates up through parent elements (target → parent → document). For example, clicking a button inside a div will fire the button's click handler and then the parent div's click handlers (unless propagation is stopped).

4. What is event delegation?

Instead of attaching listeners to many child elements, attach a single listener to a common parent and use the event's `target` to determine which child triggered it. This reduces memory usage, simplifies code, and works for dynamically added elements.

5. How does `onclick` differ from `addEventListener`?

`onclick` is an HTML/DOM property that can only hold one handler — assigning a new handler replaces the old. `addEventListener` lets you attach multiple handlers and control options like capture and once. `addEventListener` is more flexible and preferred.

6. What is the event object and what does it contain?

When an event fires, the handler receives an `event` object containing data about the event: `type`, `target` (the element that triggered it), `currentTarget`, mouse

coordinates, key pressed, `preventDefault()`, `stopPropagation()`, and other properties depending on the event type. Use it to inspect or control event behavior.

7. What is DOMContentLoaded?

A browser event fired when the initial HTML document has been completely loaded and parsed, before stylesheets/images/external resources finish. It's useful to run JS that manipulates DOM elements without waiting for everything (use `window.onload` if you need images/resources to finish).

8. Explain callbacks with an example.

A callback is a function passed into another function to run later. Example:

```js
function doWork(callback) {
  // heavy work
  callback();
}
doWork(function() { console.log('done'); });
```

Callbacks enable asynchronous control flow (but can lead to nested "callback hell"; Promises/async-await are modern alternatives).

9. How do timers (`setTimeout`, `setInterval`) work in JavaScript?

`setTimeout(fn, ms)` schedules `fn` to run once after `ms` milliseconds. `setInterval(fn, ms)` schedules repeated execution every `ms` ms until cleared with `clearInterval()`. Note: timers are handled by the event loop — long-running tasks can delay callbacks.

10. What are arrow functions and how do they differ from regular functions?

Arrow functions (`() => {}`) offer concise syntax and **lexical** `this`, meaning they do not have their own `this` binding — `this` is inherited from the surrounding scope. They also lack `arguments`, cannot be used as constructors (`new`), and are ideal for short callbacks.

✅ Assignment 5 — Detailed Answers (PHP + MySQL)

1. What is the purpose of PHP in web development?

PHP is a server-side scripting language used to generate dynamic HTML, process form data, manage sessions, access databases, and interact with the filesystem. It runs on the server and sends rendered HTML/JSON back to the client, enabling dynamic web apps.

2. Explain how a form sends data to a PHP script.

A form has `action` (target URL) and `method` (`GET` or `POST`). When submitted, the browser sends form fields to the server: `GET` appends them to the URL as query parameters, `POST` sends them in the request body. In PHP, `$_GET` or `$_POST` superglobals read submitted values.

3. What is the difference between `GET` and `POST`?

`GET` appends parameters to the URL, is idempotent and cacheable, and limited in length; it's suitable for retrieving data. `POST` sends data in the request body, not shown in the URL, suitable for creating/updating resources and larger payloads. Sensitive data should use POST (and HTTPS) to avoid exposure in logs/bookmarks.

4. How do you connect PHP to a MySQL database?

Two common ways:

- `mysqli` extension:

```php
$conn = mysqli_connect($host, $user, $pass, $db);
```

- PDO (PHP Data Objects) — preferred for prepared statements and portability:

```php
$pdo = new PDO('mysql:host=host;dbname=db', $user, $pass);
```

Handle connection errors and never expose credentials in public code.

5. What are SQL injection attacks and how do you prevent them?

SQL injection occurs when untrusted input is concatenated into SQL queries, letting attackers modify queries. **Prevention:** use parameterized queries / prepared statements (PDO or mysqli prepared statements), validate/sanitize inputs where appropriate, and follow least-privilege for DB accounts. Example (PDO):

```php
$stmt = $pdo->prepare('INSERT INTO users (name) VALUES (?)');
$stmt->execute([$name]);
```

6. Explain how `$_POST` and `$_GET` work.

They are PHP superglobal associative arrays that contain form input values from `POST` and `GET` requests respectively. Access fields like `$_POST['username']`. Always validate and sanitize these values before use.

7. What is server-side validation?

Server-side validation is checking and sanitizing inputs on the server to ensure correctness, security, and integrity of data — e.g., type checks, length limits, required fields, and format validations. Since client-side validation can be bypassed, server-side validation is mandatory.

8. What is a primary key in MySQL?

A primary key uniquely identifies each row in a table and usually has uniqueness and non-null constraints. It can be a single column or composite. Commonly implemented with an `AUTO_INCREMENT` integer for efficiency.

9. How do you insert form data into a database using PHP?

Example with PDO prepared statements:

```php
$name = $_POST['name'];
$email = $_POST['email'];
$stmt = $pdo->prepare('INSERT INTO users (name,email) VALUES (?, ?)');
$stmt->execute([$name, $email]);
```

Validate/sanitize inputs, handle errors, and use transactions for atomic operations if needed.

10. What are prepared statements and why are they important?

Prepared statements separate SQL code from data. You define a query template with placeholders, then bind/execute with data. Database engines can pre-compile the

query and safely escape values, preventing SQL injection and often improving performance for repeated queries.

✅ Want me to do any of the following next?

- Convert all of the above into a printable **PDF** or study sheet.
- Generate short **one-line answers** for quick revision cards.
- Make a **mock viva** where I ask questions and you answer interactively.
- Review any specific file(s) from the ZIP and give code-level feedback.

Tell me which of the four above (or another option) you'd like — I'll produce it right away.